SQL Retail Sales Analysis Project (120+ Queries)

```
CREATE TABLE customers (
customer_id NUMBER,
customer_name VARCHAR2(50)
);
CREATE TABLE stores (
store_id NUMBER,
store_name VARCHAR2(50)
);
CREATE TABLE products (
product_id NUMBER,
category VARCHAR2(50)
);
CREATE TABLE sales (
sale_id NUMBER,
customer_id NUMBER,
store_id NUMBER,
product_id NUMBER,
sale_date DATE,
amount NUMBER,
quantity NUMBER,
discount NUMBER
);
```

```
INSERT INTO customers
SELECT LEVEL, 'Cust_'||LEVEL FROM dual CONNECT BY LEVEL<=50;

INSERT INTO stores VALUES (1,'CMR');
INSERT INTO stores VALUES (2,'SR');
INSERT INTO stores VALUES (3,'Lucky');

INSERT INTO products VALUES (1,'Electronics');
INSERT INTO products VALUES (2,'Fashion');
INSERT INTO products VALUES (3,'Grocery');
INSERT INTO products VALUES (4,'Home');

INSERT INTO sales
SELECT
LEVEL,
MOD(LEVEL,50)+1,
MOD(LEVEL,3)+1,
MOD(LEVEL,4)+1,
DATE '2025-01-01' + MOD(LEVEL,90),
ROUND(DBMS_RANDOM.VALUE(100,10000)),
MOD(LEVEL,5)+1,
CASE WHEN MOD(LEVEL,4)=0 THEN NULL ELSE ROUND(DBMS_RANDOM.VALUE(10,500))
END
FROM dual
CONNECT BY LEVEL <= 1000;
```

Data Understanding:

--1. Find total number of transactions.

select count(*) as Total_Transactions from sales;

	TOTAL_TRANSACTIONS
1	1000

-- 2. Find total revenue generated.

select sum(amount) as Total_Revenue from sales;

	TOTAL_REVENUE
1	5116244

-- 3. Find total unique customers.

select count(distinct customer_id) as Unique_customers from sales;

	UNIQUE_CUSTOMERS
1	50

-- 4. Find average order value.

select avg(amount) as Avg_Order_Val from sales;

	AVG_ORDER_VAL
1	5116.244

-- 5. Find total quantity sold.

select sum(quantity) as Total_Quantity from sales;

	TOTAL_QUANTITY
1	3000

-- 6. Find minimum and maximum sale amount.

select min(amount) as minimum_sale ,max(amount) as maximum_sale from sales;

	MINIMUM_SALE	MAXIMUM_SALE
1	105	9981

-- 7. Find average discount given.

select avg(discount) as Avg_Discount from sales;

	AVG_DISCOUNT
1	251.764

-- 8. Find total sales per store.

select store_id,sum(amount) as Total_Sale from sales group by store_id order by store_id

	STORE_ID	TOTAL_SALE
1	1	1691808
2	2	1719595
3	3	1704841

-- 9. Find total sales per category.

select product_id,sum(amount) as Total_sales from sales group by product_id order by product_id

	STORE_ID	TOTAL_SALE
1	1	1691808
2	2	1719595
3	3	1704841

-- 10. Find number of transactions per day.

select sale_date,count(*) as Transactions from sales group by sale_date order by sale_date;

	SALE_DATE	TRANSACTIONS
1	1/1/2025, 12:00:00	11
2	1/2/2025, 12:00:00	12
3	1/3/2025, 12:00:00	12
4	1/4/2025, 12:00:00	12
5	1/5/2025, 12:00:00	12

Data Cleaning

-- 11. Replace NULL discount with 0.

select nvl(discount,0) as discount from sales;

	DISCOUNT
1	396
2	49
3	0
4	245
5	55

-- 12. Find number of NULL discounts.

select count(*) as Null_Discount from sales where discount is null;

	NULL_DISCOUNT
1	250

-- 13. Calculate net revenue (amount - discount).

select (amount-nvl(discount,0)) as Net_Revenue from sales;

	NET_REVENUE
1	6432
2	3018
3	2104
4	7396
5	1196

-- 14. Replace NULL quantity with 1.

select nvl(quantity,1) as Quantity from sales;

	QUANTITY
1	1
2	2
3	3
4	4
5	5

-- 15. Identify rows where amount is NULL.

select * from sales where amount is null;

-- 16. Use COALESCE to handle multiple NULL columns.

select SALE_ID,COALESCE(discount,0) as discount from sales;

	SALE_ID	DISCOUNT
1	610	396
2	611	49
3	612	0
4	613	245
5	614	55

-- 17. Create a cleaned column for revenue.

SELECT sale_id,DISCOUNT,AMOUNT,

NVL2(discount, amount - discount, amount) AS cleaned_revenue

FROM sales;

	SALE_ID	DISCOUNT	AMOUNT	CLEANED_REVENUE
1	610	396	6828	6432
2	611	49	3067	3018
3	612	*(null)*	2104	2104
4	613	245	7641	7396
5	614	55	1251	1196

-- 18. Check percentage of missing data.

SELECT ROUND((COUNT(CASE WHEN amount IS NULL THEN 1 END) / COUNT(*))*100,2) AS amount_missing,

ROUND((COUNT(CASE WHEN discount IS NULL THEN 1 END) / COUNT(*))*100,2) AS discount_missing,

ROUND((COUNT(CASE WHEN quantity IS NULL THEN 1 END) / COUNT(*))*100,2) AS quantity_missing FROM sales;

	AMOUNT_MISSING	DISCOUNT_MISSING	QUANTITY_MISSING
1	0	25	0

-- 19. Flag rows with missing values.

select sale_id,amount,DISCOUNT,quantity,case when amount is null or discount is null or quantity is null then 'Missing' else 'Complete' end as Flag from sales;

	SALE_ID	AMOUNT	DISCOUNT	QUANTITY	FLAG
1	610	6828	396	1	Complete
2	611	3067	49	2	Complete
3	612	2104	*(null)*	3	Missing
4	613	7641	245	4	Complete
5	614	1251	55	5	Complete

Filtering (WHERE)

-- 21. Find sales in last 30 days.

select * from sales where sale_date>=(select max(sale_date))-30;

	SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
1	610	11	2	3	3/12/2025, 12:00:0	6828	1	396
2	611	12	3	4	3/13/2025, 12:00:0	3067	2	49
3	612	13	1	1	3/14/2025, 12:00:0	2104	3	*(null)*
4	613	14	2	2	3/15/2025, 12:00:0	7641	4	245
5	614	15	3	3	3/16/2025, 12:00:0	1251	5	55

-- 22. Find sales above 5000.

select * from sales where amount>5000;

	SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
1	610	11	2	3	3/12/2025, 12:00:0	6828	1	396
2	613	14	2	2	3/15/2025, 12:00:0	7641	4	245
3	616	17	2	1	3/18/2025, 12:00:0	8335	2	*(null)*
4	618	19	1	3	3/20/2025, 12:00:0	9627	4	402
5	619	20	2	4	3/21/2025, 12:00:0	6701	5	49

-- 23. Find sales between 1000 and 5000.

select * from sales where amount>1000 and amount<5000

	SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
1	611	12	3	4	3/13/2025, 12:00:0	3067	2	49
2	612	13	1	1	3/14/2025, 12:00:0	2104	3	*(null)*
3	614	15	3	3	3/16/2025, 12:00:0	1251	5	55
4	615	16	1	4	3/17/2025, 12:00:0	2304	1	275
5	623	24	3	4	3/25/2025, 12:00:0	2919	4	58

-- 24. Find sales from CMR store.

select sa.* from sales sa join stores s on sa.store_id=s.store_id where s.store_name='CMR'

	SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
1	612	13	1	1	3/14/2025, 12:00:0	2104	3	(null)
2	615	16	1	4	3/17/2025, 12:00:0	2304	1	275
3	618	19	1	3	3/20/2025, 12:00:0	9627	4	402
4	621	22	1	2	3/23/2025, 12:00:0	9396	2	103
5	624	25	1	1	3/26/2025, 12:00:0	2754	5	(null)

-- 25. Find Electronics category sales.

select sa.* from sales sa join products p on sa.product_id=p.product_id where p.category='Electronics'

	SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
1	612	13	1	1	3/14/2025, 12:00:0	2104	3	(null)
2	616	17	2	1	3/18/2025, 12:00:0	8335	2	(null)
3	620	21	3	1	3/22/2025, 12:00:0	6272	1	(null)
4	624	25	1	1	3/26/2025, 12:00:0	2754	5	(null)
5	628	29	2	1	3/30/2025, 12:00:0	6239	4	(null)

-- 26. Find sales excluding Grocery category.

select sa.* from sales sa join products p on sa.product_id=p.product_id where p.category<>'Grocery'

	SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
1	611	12	3	4	3/13/2025, 12:00:0	3067	2	49
2	612	13	1	1	3/14/2025, 12:00:0	2104	3	(null)
3	613	14	2	2	3/15/2025, 12:00:0	7641	4	245
4	615	16	1	4	3/17/2025, 12:00:0	2304	1	275
5	616	17	2	1	3/18/2025, 12:00:0	8335	2	(null)

-- 27. Find customers from specific store.

select c.customer_name,s.store_name from customers c join sales sa on c.customer_id=sa.customer_id join stores s on sa.store_id=s.store_id group by c.customer_name,s.store_name

	CUSTOMER_NAME	STORE_NAME
1	Cust_11	SR
2	Cust_12	Lucky
3	Cust_13	CMR
4	Cust_14	SR
5	Cust_15	Lucky

-- 28. Find high discount transactions.

select * from sales where discount>300

	SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
1	610	11	2	3	3/12/2025, 12:00:0	6828	1	396
2	618	19	1	3	3/20/2025, 12:00:0	9627	4	402
3	622	23	2	3	3/24/2025, 12:00:0	7367	3	404
4	625	26	2	2	3/27/2025, 12:00:0	6847	1	399
5	627	28	1	4	3/29/2025, 12:00:0	1986	3	393

-- 29. Find low quantity sales.

select * from sales where quantity<2

	SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
1	610	11	2	3	3/12/2025, 12:00:0	6828	1	396
2	615	16	1	4	3/17/2025, 12:00:0	2304	1	275
3	620	21	3	1	3/22/2025, 12:00:0	6272	1	*(null)*
4	625	26	2	2	3/27/2025, 12:00:0	6847	1	399
5	630	31	1	3	1/1/2025, 12:00:00	1917	1	175

-- 30. Find recent transactions.

select * from sales order by sale_date desc fetch first 10 rows only;

	SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
1	629	30	3	2	3/31/2025, 12:00:0	3072	5	107
2	449	50	3	2	3/31/2025, 12:00:0	8222	5	458
3	359	10	3	4	3/31/2025, 12:00:0	2820	5	251
4	269	20	3	2	3/31/2025, 12:00:0	2051	5	62
5	179	30	3	4	3/31/2025, 12:00:0	6833	5	194

Aggregation (GROUP BY)

-- 31. Revenue per store.

select s.store_name,sum(sa.amount) as Revenue from stores s join sales sa on s.store_id=sa.store_id group by s.store_name

	STORE_NAME	REVENUE
1	SR	1719595
2	Lucky	1704841
3	CMR	1691808

-- 32. Revenue per category.

select p.category,sum(sa.amount) as Revenue from products p join sales sa on p.product_id=sa.product_id group by p.category

	CATEGORY	REVENUE
1	Grocery	1270860
2	Home	1220854
3	Electronics	1242652
4	Fashion	1381878

-- 33. Revenue per customer.

select c.customer_name,sum(sa.amount) as Revenue from customers c join sales sa on c.customer_id=sa.customer_id group by c.customer_name

	CUSTOMER_NAME	REVENUE
1	Cust_11	93577
2	Cust_12	113348
3	Cust_13	93714
4	Cust_14	116655
5	Cust_15	72960

-- 34. Monthly revenue.

select to_char(sale_date,'mm') as month,sum(amount) as monthly_revenue from sales group by to_char(sale_date,'mm') order by month

	MONTH	MONTHLY_REVENU
1	01	1772396
2	02	1568236
3	03	1775612

-- 35. Daily revenue.

select sale_date,sum(amount) from sales group by sale_date order by sale_date;

	SALE_DATE	SUM(AMOUNT)
1	1/1/2025, 12:00:00	54543
2	1/2/2025, 12:00:00	63056
3	1/3/2025, 12:00:00	64701
4	1/4/2025, 12:00:00	71214
5	1/5/2025, 12:00:00	51742

-- 36. Average sales per store.

select s.store_name,round(avg(nvl(sa.amount,0)),2) as Avg_sales from sales sa join stores s on sa.store_id=s.store_id group by s.store_name;

	STORE_NAME	AVG_SALES
1	SR	5148.49
2	Lucky	5119.64
3	CMR	5080.5

-- 37. Total quantity per category.

select p.category,sum(nvl(sa.quantity,0)) as Total_quantity from products p join sales sa on p.product_id=sa.product_id group by p.category;

	CATEGORY	TOTAL_QUANTITY
1	Grocery	750
2	Home	750
3	Electronics	750
4	Fashion	750

-- 38. Total discount per store.

select s.store_name,sum(nvl(sa.discount,0)) as Total_discount from stores s join sales sa on s.store_id=sa.store_id group by s.store_name;

	STORE_NAME	TOTAL_DISCOUNT
1	SR	64623
2	Lucky	62710
3	CMR	61490

-- 39. Customer-wise total spending.

select c.customer_name,sum(sa.amount) as Total_spending from customers c join sales sa on c.customer_id=sa.customer_id group by c.customer_name

	CUSTOMER_NAME	TOTAL_SPENDING
1	Cust_11	93577
2	Cust_12	113348
3	Cust_13	93714
4	Cust_14	116655
5	Cust_15	72960

-- 40. Store-wise transaction count.

select s.store_name,count(*) as Transaction_count from stores s join sales sa on s.store_id=sa.store_id group by s.store_name

	STORE_NAME	TRANSACTION_COUNT
1	SR	334
2	Lucky	333
3	CMR	333

HAVING:

-- 41. Customers with revenue > 20000.

select c.customer_name,amount from customers c join sales sa on c.customer_id=sa.customer_id where amount>2000 order by amount

	CUSTOMER_NAME	AMOUNT
1	Cust_38	2015
2	Cust_31	2022
3	Cust_3	2027
4	Cust_33	2030
5	Cust_13	2038

-- 42. Stores with revenue > 100000.

select s.store_name,sum(amount) as Revenue from stores s join sales sa on s.store_id=sa.store_id group by s.store_name having sum(amount)>100000

	STORE_NAME	REVENUE
1	SR	1719595
2	Lucky	1704841
3	CMR	1691808

-- 43. Categories with avg sales > 3000.

select p.category,avg(amount) as Avg_sales from products p join sales sa on p.product_id=sa.product_id group by p.category having avg(amount)>3000;

	CATEGORY	AVG_SALES
1	Grocery	5083.44
2	Home	4883.416
3	Electronics	4970.608
4	Fashion	5527.512

-- 44. Customers with more than 5 transactions.

select c.customer_name,count(*) as Transactions from customers c join sales s on c.customer_id=s.customer_id group by c.customer_name having count(*)>5

	CUSTOMER_NAME	TRANSACTIONS
1	Cust_11	20
2	Cust_12	20
3	Cust_13	20
4	Cust_14	20
5	Cust_15	20

-- 45. Stores with high discount usage.

select s.store_name,sum(nvl(discount,0)) as Total_discount from stores s join sales sa on s.store_id=sa.store_id group by store_name having sum(nvl(discount,0))>5000;

	STORE_NAME	TOTAL_DISCOUNT
1	SR	64623
2	Lucky	62710
3	CMR	61490

-- 46. Categories with low performance.

select p.category,sum(nvl(amount,0)) as Total_revenue from products p join sales sa on p.product_id=sa.product_id group by p.category having sum(nvl(amount,0))<1300000;

	CATEGORY	TOTAL_REVENUE
1	Grocery	1270860
2	Home	1220854
3	Electronics	1242652

-- 47. Customers with high avg order value.

select c.customer_name,avg(nvl(sa.amount,0)) as Avg_order from customers c join sales sa on c.customer_id=sa.customer_id group by c.customer_name having avg(nvl(amount,0))>5000

	CUSTOMER_NAME	AVG_ORDER
1	Cust_12	5667.4
2	Cust_14	5832.75
3	Cust_16	5678.4
4	Cust_19	5213.95
5	Cust_20	5780.6

-- 48. Find top 5 customers using HAVING.

select c.customer_name,sum(amount) as Total_Revenue from customers c join sales sa on c.customer_id=sa.customer_id group by c.customer_name order by Total_Revenue desc fetch first 5 row only;

	CUSTOMER_NAME	TOTAL_REVENUE
1	Cust_29	131065
2	Cust_39	129528
3	Cust_36	124893
4	Cust_6	121601
5	Cust_26	117830

-- 49. Stores with max transactions.

select s.store_name,count(*) as Max_Transactions from stores s join sales sa on s.store_id=sa.store_id group by s.store_name order by Max_Transactions desc fetch first 1 row only;

	STORE_NAME	MAX_TRANSACTIONS
1	SR	334

-- 50. Categories with high growth.

select p.category,sum(sa.amount) as Total_Revenue from products p join sales sa on p.product_id=sa.product_id group by p.category order by Total_Revenue desc;

	CATEGORY	TOTAL_REVENUE
1	Fashion	1381878
2	Grocery	1270860
3	Electronics	1242652
4	Home	1220854

CASE WHEN

-- 51. Classify customers as High/Medium/Low spenders.

```
SELECT c.customer_id,
    SUM(sa.amount) AS total_Revenue,
    CASE
      WHEN SUM(sa.amount) > 100000 THEN 'High Spender'
      WHEN SUM(sa.amount) BETWEEN 90000 AND 100000 THEN 'Medium Spender'
      ELSE 'Low Spender'
    END AS Spenders
FROM sales sa
JOIN customers c ON sa.customer_id = c.customer_id
GROUP BY c.customer_id;
```

	CUSTOMER_ID	TOTAL_REVENUE	SPENDERS
1	11	93577	Medium Spender
2	12	113348	High Spender
3	13	93714	Medium Spender
4	14	116655	High Spender
5	15	72960	Low Spender

-- 52. Classify transactions as Big/Small.

```
SELECT sale_id, amount,
    CASE
      WHEN amount > 5000 THEN 'Big Transaction'
      ELSE 'Small Transaction'
    END AS Transaction_Type
FROM sales;
```

	SALE_ID	AMOUNT	TRANSACTION_TYPE
1	610	6828	Big Transaction
2	611	3067	Small Transaction
3	612	2104	Small Transaction
4	613	7641	Big Transaction
5	614	1251	Small Transaction

-- 53. Create discount flag.

```
SELECT sale_id, nvl(discount,0),
    CASE
      WHEN NVL(discount,0) > 0 THEN 'Discount Applied'
      ELSE 'No Discount'
    END AS discount_flag
FROM sales;
```

	SALE_ID	NVL(DISCOUNT,0)	DISCOUNT_FLAG
1	610	396	Discount Applied
2	611	49	Discount Applied
3	612	0	No Discount
4	613	245	Discount Applied
5	614	55	Discount Applied

-- 54. Segment stores based on revenue.

```
select s.store_name,sum(sa.amount)  as Total_Revenue,
case when sum(sa.amount)>1710000 then 'High_Revenue'
when sum(sa.amount) between 1700000 and 1710000 then 'Medium Revenue'
else 'Low Revenue' end as Store_segment
from stores s join sales sa on s.store_id=sa.store_id group by s.store_name
```

	STORE_NAME	TOTAL_REVENUE	STORE_SEGMENT
1	SR	1719595	High_Revenue
2	Lucky	1704841	Medium Revenue
3	CMR	1691808	Low Revenue

-- 55. Categorize customers based on frequency.

```
select c.customer_name,count(*) as Transactions,
case when count(*)>20 then 'Frequent Buyer'
when count(*) between 10 and 20 then 'Moderate Buyer'
else 'Ocassional_buyer' end as Frequency from customers c join sales sa
on c.customer_id=sa.customer_id group by c.customer_name;
```

	CUSTOMER_NAME	TRANSACTIONS	FREQUENCY
1	Cust_11	20	Moderate Buyer
2	Cust_12	20	Moderate Buyer
3	Cust_13	20	Moderate Buyer
4	Cust_14	20	Moderate Buyer
5	Cust_15	20	Moderate Buyer

-- 56. Create sales buckets.

```
select sale_id,amount,
case when amount>6000 then 'Bucket A'
when amount between 2000 and 5000 then 'Bucket B'
else 'Bucket C' end as Buckets from sales;
```

	SALE_ID	AMOUNT	BUCKETS
1	610	6828	Bucket A
2	611	3067	Bucket B
3	612	2104	Bucket B
4	613	7641	Bucket A
5	614	1251	Bucket C

-- 57. Identify premium customers.

select c.customer_name,sum(nvl(sa.amount,0)) as Total_Revenue,

case when sum(nvl(sa.amount,0))>100000 then 'Premium Customers'

else 'Regular customers' end as customer_type from customers c join sales sa on c.customer_id=sa.customer_id group by c.customer_name

	CUSTOMER_NAME	TOTAL_REVENUE	CUSTOMER_TYPE
1	Cust_11	93577	Regular customers
2	Cust_12	113348	Premium Customers
3	Cust_13	93714	Regular customers
4	Cust_14	116655	Premium Customers
5	Cust_15	72960	Regular customers

-- 58. Mark high-value transactions.

select sale_id,amount,

case when amount>5000 then 'High Value'

else 'Normal' end as Transaction_type from sales

	SALE_ID	AMOUNT	TRANSACTION_TYPE
1	610	6828	High Value
2	611	3067	Normal
3	612	2104	Normal
4	613	7641	High Value
5	614	1251	Normal

-- 59. Classify categories based on performance.

select p.category,sum(sa.amount),

case when sum(sa.amount)>1300000 then 'Top performing'

when sum(sa.amount) between 1240000 and 1300000 then 'Medium performing'

else 'Low performing' end as performance from products p join sales sa on p.product_id=sa.product_id group by p.category

	CATEGORY	SUM(SA.AMOUNT)	PERFORMANCE
1	Grocery	1270860	Medium performing
2	Home	1220854	Low performing
3	Electronics	1242652	Medium performing
4	Fashion	1381878	Top performing

-- 60. Create custom labels for reporting.

select sale_id,amount,discount,quantity,

case when amount>5000 and discount>300 then 'VIP Deal'

when amount between 3000 and 5000 then 'Normal Deal'

else 'Basic Deal' end as Deals from sales;

	SALE_ID	AMOUNT	DISCOUNT	QUANTITY	DEALS
1	610	6828	396	1	VIP Deal
2	611	3067	49	2	Normal Deal
3	612	2104	*(null)*	3	Basic Deal
4	613	7641	245	4	Basic Deal
5	614	1251	55	5	Basic Deal

JOINS (Multiple Tables)

-- 61. Join sales with customers table.

select sa.sale_id,amount,c.customer_id,c.CUSTOMER_NAME from sales sa join customers c on sa.customer_id=c.customer_id;

	SALE_ID	AMOUNT	CUSTOMER_ID	CUSTOMER_NAME
1	610	6828	11	Cust_11
2	611	3067	12	Cust_12
3	612	2104	13	Cust_13
4	613	7641	14	Cust_14
5	614	1251	15	Cust_15

-- 62. Join sales with stores table.

select sa.sale_id,amount,s.store_id,s.store_name from sales sa join stores s on sa.store_id=s.store_id;

	SALE_ID	AMOUNT	STORE_ID	STORE_NAME
1	610	6828	2	SR
2	611	3067	3	Lucky
3	612	2104	1	CMR
4	613	7641	2	SR
5	614	1251	3	Lucky

-- 63. Join sales with products table.

select sa.sale_id,amount,p.product_id,p.category from sales sa join products p on sa.product_id=p.product_id;

	SALE_ID	AMOUNT	PRODUCT_ID	CATEGORY
1	610	6828	3	Grocery
2	611	3067	4	Home
3	612	2104	1	Electronics
4	613	7641	2	Fashion
5	614	1251	3	Grocery

-- 64. Find customer name with store name and sales.

select c.customer_name,s.store_name,sa.amount from sales sa join stores s on sa.store_id=s.store_id join customers c on c.customer_id=sa.customer_id

	CUSTOMER_NAME	STORE_NAME	AMOUNT
1	Cust_11	SR	6828
2	Cust_12	Lucky	3067
3	Cust_13	CMR	2104
4	Cust_14	SR	7641
5	Cust_15	Lucky	1251

-- 65. Find category-wise revenue using joins.

select p.category,sum(sa.amount) as Total_Revenue from sales sa join products p on sa.product_id=p.product_id group by p.category

	CATEGORY	TOTAL_REVENUE
1	Grocery	1270860
2	Home	1220854
3	Electronics	1242652
4	Fashion	1381878

-- 66. Find store-wise customer count.

select s.store_name,count(sa.customer_id) as customer from sales sa join stores s on sa.store_id=s.store_id join customers c on c.customer_id=sa.customer_id group by s.store_name;

	STORE_NAME	CUSTOMER
1	SR	334
2	Lucky	333
3	CMR	333

-- 67. Find top product per store.

select s.store_name,p.product_id,p.category from sales sa join stores s on sa.store_id=s.store_id join products p on p.product_id=sa.product_id group by s.store_name ,p.product_id,p.category

	STORE_NAME	PRODUCT_ID	CATEGORY
1	SR	3	Grocery
2	Lucky	4	Home
3	CMR	1	Electronics
4	SR	2	Fashion
5	Lucky	3	Grocery

-- 68. Find customers who purchased from multiple stores.

select c.customer_name,count(distinct sa.store_id) from sales sa join customers c on sa.customer_id=c.customer_id group by c.customer_name having count(store_id)>1

	CUSTOMER_NAME	COUNT(DISTINCTSA.STORE_ID)
1	Cust_11	3
2	Cust_12	3
3	Cust_13	3
4	Cust_14	3
5	Cust_15	3

-- 69. Find sales with product category.

select p.category,sum(sa.amount) from sales sa join products p on sa.product_id=p.product_id group by p.category

	CATEGORY	SUM(SA.AMOUNT)
1	Grocery	1270860
2	Home	1220854
3	Electronics	1242652
4	Fashion	1381878

-- 70. Find revenue per customer per store.

select c.customer_name,s.store_name,sum(sa.amount) from sales sa

join customers c on sa.customer_id=c.customer_id

join stores s on sa.store_id=s.store_id

group by c.customer_name,s.store_name order by customer_name,store_name

	CUSTOMER_NAME	STORE_NAME	SUM(SA.AMOUNT)
1	Cust_1	CMR	36009
2	Cust_1	Lucky	33301
3	Cust_1	SR	26909
4	Cust_10	CMR	22719
5	Cust_10	Lucky	32444

-- 71. Find customers with no transactions (LEFT JOIN).

select c.customer_name,sa.sale_id from sales sa left join customers c on sa.customer_id=c.customer_id where sale_id is null

-- 72. Find missing product mappings.

select sa.product_id from sales sa join products p on p.product_id=sa.product_id where sa.product_id is null

-- 73. Find store performance using joins.

select s.store_name,sum(sa.amount) as Total_Revenue,count(*) as Transactions from sales sa join stores s on sa.store_id=s.store_id group by s.store_name;

	STORE_NAME	TOTAL_REVENUE	TRANSACTIONS
1	SR	1719595	334
2	Lucky	1704841	333
3	CMR	1691808	333

-- 74. Find cross-category purchases.

WITH category_pairs AS (

SELECT p1.category AS category1,p2.category AS category2,COUNT(*) AS purchase_count FROM sales sa1 JOIN sales sa2

ON sa1.sale_id <> sa2.sale_id AND sa1.customer_id = sa2.customer_id AND sa1.sale_date = sa2.sale_date AND sa1.product_id < sa2.product_id

JOIN products p1 ON sa1.product_id = p1.product_id JOIN products p2 ON sa2.product_id = p2.product_id GROUP BY p1.category, p2.category

)

SELECT category1, category2, purchase_count

FROM category_pairs

ORDER BY purchase_count DESC;

	CATEGORY1	CATEGORY2	PURCHASE_COUNT
1	Fashion	Home	275
2	Electronics	Grocery	275

-- 75. Find total revenue using joins.

select sum(sa.amount) as Total_Revenue from sales sa join customers c on sa.customer_id=c.customer_id join products p on sa.product_id=p.product_id join stores s on sa.store_id=s.store_id;

	TOTAL_REVENUE
1	5116244

Window Functions

-- 76. Calculate total spend per customer using window function.

select distinct c.customer_name,sum(sa.amount) over(Partition by c.customer_name) as Total_spend

from sales sa join customers c on sa.customer_id=c.customer_id

	CUSTOMER_NAME	TOTAL_SPEND
1	Cust_1	96219
2	Cust_10	99997
3	Cust_11	93577
4	Cust_12	113348
5	Cust_13	93714

-- 77. Rank customers by revenue.

select c.customer_name,sum(sa.amount) as Revenue ,

rank() over(order by sum(sa.amount) desc) as Rank

from sales sa join customers c on sa.customer_id=c.customer_id group by c.customer_name

	CUSTOMER_NAME	REVENUE	RANK
1	Cust_29	131065	1
2	Cust_39	129528	2
3	Cust_36	124893	3
4	Cust_6	121601	4
5	Cust_26	117830	5

-- 78. Find top customer per store.

select store_name,customer_name,Total_Revenue from

(select c.customer_name,s.store_name,sum(sa.amount) as Total_Revenue,

rank() over(partition by s.store_name order by sum(sa.amount) desc) as rnk from sales sa

join customers c on sa.customer_id=c.customer_id join stores s

on sa.store_id=s.store_id group by c.customer_name,s.store_name)

where rnk=1

	STORE_NAME	CUSTOMER_NAME	TOTAL_REVENUE
1	CMR	Cust_29	53337
2	Lucky	Cust_39	54938
3	SR	Cust_6	48496

-- 79. Calculate running total of sales.

select sale_id,amount,sum(amount) over(order by sale_id) as Running_Total from sales

	SALE_ID	AMOUNT	RUNNING_TOTAL
1	1	9437	9437
2	2	5171	14608
3	3	5928	20536
4	4	9428	29964
5	5	9573	39537

-- 80. Find previous sale using LAG.

select sale_id,amount,lag(amount) over(order by sale_id) as Lag from sales;

	SALE_ID	AMOUNT	LAG
1	1	9437	*(null)*
2	2	5171	9437
3	3	5928	5171
4	4	9428	5928
5	5	9573	9428

-- 81. Find next sale using LEAD.

select sale_id,amount,lead(amount) over(order by sale_id) as Lead from sales;

	SALE_ID	AMOUNT	LEAD
1	1	9437	5171
2	2	5171	5928
3	3	5928	9428
4	4	9428	9573
5	5	9573	3406

-- 82. Calculate sales growth.

select sale_id,amount,lag(amount) over(order by sale_id) as Lag,

amount-lag(amount) over(order by sale_id) as Growth

from sales;

	SALE_ID	AMOUNT	LAG	GROWTH
1	1	9437	(null)	(null)
2	2	5171	9437	-4266
3	3	5928	5171	757
4	4	9428	5928	3500
5	5	9573	9428	145

-- 83. Find difference between transactions.

select sale_id,amount,lag(amount) over(order by sale_id) as Lag,

amount-lag(amount) over(order by sale_id) as Diff_Bet_Trans

from sales;

	SALE_ID	AMOUNT	LAG	DIFF_BET_TRANS
1	1	9437	(null)	(null)
2	2	5171	9437	-4266
3	3	5928	5171	757
4	4	9428	5928	3500
5	5	9573	9428	145

-- 84. Calculate cumulative revenue.

select sale_id,amount,sum(amount) over(order by sale_id rows between unbounded preceding and current row) as Cumulative_Revenue from sales;

	SALE_ID	AMOUNT	CUMULATIVE_REVENUE
1	1	9437	9437
2	2	5171	14608
3	3	5928	20536
4	4	9428	29964
5	5	9573	39537

-- 85. Find moving average of sales.

select sale_id,amount,round(avg(amount) over(order by sale_id rows between current row and 2 following),2) as Moving_avg from sales;

	SALE_ID	AMOUNT	MOVING_AVG
1	1	9437	6845.33
2	2	5171	6842.33
3	3	5928	8309.67
4	4	9428	7469
5	5	9573	5923.33

-- 86. Rank stores by revenue.

select s.store_name,sum(sa.amount) as revenue, rank() over(order by sum(sa.amount) desc) as Store_rank

from sales sa join stores s on sa.store_id=s.store_id group by s.store_name;

	STORE_NAME	REVENUE	STORE_RANK
1	SR	1719595	1
2	Lucky	1704841	2
3	CMR	1691808	3

-- 87. Find second highest sale.

SELECT sale_id, amount

FROM (SELECT sale_id, amount,

RANK() OVER (ORDER BY amount DESC) AS rnk FROM sales)

WHERE rnk = 2;

	SALE_ID	AMOUNT
1	559	9976

-- 88. Partition data by store and analyze.

select s.store_name,sa.amount,sa.sale_id,sum(sa.amount) over(partition by s.store_name order by sale_id)

as Data from sales sa join stores s on sa.store_id=s.store_id;

	STORE_NAME	AMOUNT	SALE_ID	DATA
1	CMR	5928	3	5928
2	CMR	3406	6	9334
3	CMR	7613	9	16947
4	CMR	5333	12	22280
5	CMR	207	15	22487

-- 89. Find customer ranking per store.

select c.customer_name,s.store_name,sum(sa.amount) as Total_revenue,rank() over(partition by s.store_name order by sum(sa.amount) desc) as rnk

from sales sa join customers c on sa.customer_id=c.customer_id join stores s

on sa.store_id=s.store_id group by c.customer_name,s.store_name,sa.amount;

	CUSTOMER_NAME	STORE_NAME	TOTAL_REVENUE	RNK
1	Cust_12	CMR	9966	1
2	Cust_29	CMR	9924	2
3	Cust_16	CMR	9852	3
4	Cust_50	CMR	9839	4

-- 90. Calculate frequency using window.

select c.customer_name,sa.sale_id,count(sa.sale_id) over(partition by c.customer_name) as Frequency

from sales sa join customers c on sa.customer_id=c.customer_id;

	CUSTOMER_NAME	SALE_ID	FREQUENCY
1	Cust_1	650	20
2	Cust_1	700	20
3	Cust_1	750	20
4	Cust_1	800	20
5	Cust_1	850	20

NTILE / Segmentation:

-- 91. Divide customers into 4 segments.

select c.customer_name,ntile(4) over(order by amount desc) as Segments from sales sa join customers c

on sa.customer_id=c.customer_id;

	CUSTOMER_NAME	SEGMENTS
1	Cust_12	1
2	Cust_10	1
3	Cust_12	1
4	Cust_11	1
5	Cust_29	1

-- 92. Identify top 25% customers.

select c.customer_name,ntile(4) over(order by amount desc) as Segments,

case when ntile(4) over(order by amount desc)=1 then 'Top 25%'

else 'Not Top 25%' end as ranking from sales sa join customers c

on sa.customer_id=c.customer_id;

	CUSTOMER_NAME	SEGMENTS	RANKING
1	Cust_12	1	Top 25%
2	Cust_10	1	Top 25%
3	Cust_12	1	Top 25%
4	Cust_11	1	Top 25%
5	Cust_29	1	Top 25%

-- 93. Identify bottom 25% customers.

```
select c.customer_name,ntile(4) over(order by amount desc) as Segments,
case when ntile(4) over(order by amount desc)=4  then 'Bottom 25%'
else 'Not Bottom 25%' end as ranking from sales sa join customers c
on sa.customer_id=c.customer_id;
```

	CUSTOMER_NAME	SEGMENTS	RANKING
1	Cust_12	1	Not Bottom 25%
2	Cust_10	1	Not Bottom 25%
3	Cust_12	1	Not Bottom 25%
4	Cust_11	1	Not Bottom 25%
5	Cust_29	1	Not Bottom 25%

-- 94. Segment stores into 3 groups.

```
select s.store_name,ntile(3) over(order by amount desc) as Segments
from sales sa join stores s on sa.store_id=s.store_id;
```

	STORE_NAME	SEGMENTS
1	Lucky	1
2	SR	1
3	CMR	1
4	Lucky	1
5	CMR	1

-- 95. Create quartiles based on revenue.

```
select c.customer_name,sum(sa.amount) as Total_Revenue,
ntile(4) over(order by sum(sa.amount) desc) as Quartiles from sales sa
join customers c on sa.customer_id=c.customer_id group by c.customer_name;
```

	CUSTOMER_NAME	TOTAL_REVENUE	QUARTILES
1	Cust_29	131065	1
2	Cust_39	129528	1
3	Cust_36	124893	1
4	Cust_6	121601	1
5	Cust_26	117830	1

-- 96. Segment customers per store.

select s.store_name,c.customer_name,sum(sa.amount) as Total_Revenue,ntile(4) over(partition by s.store_name order by sum(sa.amount) desc) as Store_segment

from sales sa join stores s on sa.store_id=s.store_id join customers c on sa.customer_id=c.customer_id group by s.store_name,c.customer_name;

	STORE_NAME	CUSTOMER_NAME	TOTAL_REVENUE	STORE_SEGMENT
1	CMR	Cust_29	53337	1
2	CMR	Cust_20	50837	1
3	CMR	Cust_22	46417	1
4	CMR	Cust_14	44512	1
5	CMR	Cust_32	44219	1

-- 97. Identify premium segment.

select c.customer_name,sum(sa.amount),ntile(4) over(order by sum(sa.amount) desc) as Segments,

case when ntile(4) over(order by sum(sa.amount) desc)=1 then 'Premium '

else 'Non Premium' end as ranking from sales sa join customers c

on sa.customer_id=c.customer_id group by c.customer_name;

	CUSTOMER_NAME	SUM(SA.AMOUNT)	SEGMENTS	RANKING
1	Cust_29	131065	1	Premium
2	Cust_39	129528	1	Premium
3	Cust_36	124893	1	Premium
4	Cust_6	121601	1	Premium
5	Cust_26	117830	1	Premium

-- 98. Find mid-level customers.

select c.customer_name,sum(sa.amount) as Total_Revenue,ntile(4) over(order by sum(sa.amount) desc) as Segments,

case when ntile(4) over(order by sum(sa.amount) desc) in(2,3) then 'Mid-Level'

else 'Not-Mid-Level' end as Ranking from sales sa join customers c on sa.customer_id=c.customer_id group by c.customer_name;

	CUSTOMER_NAME	TOTAL_REVENUE	SEGMENTS	RANKING
1	Cust_29	131065	1	Not-Mid-Level
2	Cust_39	129528	1	Not-Mid-Level
3	Cust_36	124893	1	Not-Mid-Level
4	Cust_6	121601	1	Not-Mid-Level
5	Cust_26	117830	1	Not-Mid-Level

-- 99. Analyze bucket distribution.

```
SELECT NTILE(4) OVER (ORDER BY SUM(sa.amount) DESC) AS quartile,COUNT(*) AS customer_count,

CASE WHEN NTILE(4) OVER (ORDER BY SUM(sa.amount) DESC) = 1 THEN 'Premium'

WHEN NTILE(4) OVER (ORDER BY SUM(sa.amount) DESC) = 2 THEN 'High'

WHEN NTILE(4) OVER (ORDER BY SUM(sa.amount) DESC) = 3 THEN 'Mid'

ELSE 'Low' END AS segment_label FROM sales sa

JOIN customers c ON sa.customer_id = c.customer_id GROUP BY c.customer_name;
```

	QUARTILE	CUSTOMER_COUNT	SEGMENT_LABEL
1	1	20	Premium
2	1	20	Premium
3	1	20	Premium
4	1	20	Premium
5	1	20	Premium

-- 100. Create marketing segments.

```
SELECT c.customer_name,SUM(sa.amount) AS total_revenue,

CASE WHEN NTILE(4) OVER (ORDER BY SUM(sa.amount) DESC) = 1 THEN 'Premium'

WHEN NTILE(4) OVER (ORDER BY SUM(sa.amount) DESC) = 2 THEN 'High'

WHEN NTILE(4) OVER (ORDER BY SUM(sa.amount) DESC) = 3 THEN 'Mid'

ELSE 'Low' END AS marketing_segment FROM sales sa

JOIN customers c ON sa.customer_id = c.customer_id GROUP BY c.customer_name;
```

	CUSTOMER_NAME	TOTAL_REVENUE	MARKETING_SEGMENT
1	Cust_29	131065	Premium
2	Cust_39	129528	Premium
3	Cust_36	124893	Premium
4	Cust_6	121601	Premium

FIRST_VALUE / LAST_VALUE

-- 101. Find highest sale.

select FIRST_VALUE(amount) over(order by amount desc) as Highest_Value from sales fetch first 1 row only;

	HIGHEST_VALUE
1	9981

-- 102. Find lowest sale.

select last_value(amount) over(order by amount desc rows between unbounded preceding and unbounded following) as Lowest_Value from sales fetch first 1 row only;

	LOWEST_VALUE
1	105

-- 103. Compare each sale with highest.

select amount,FIRST_VALUE(amount) over(order by amount desc) as Highest_Value from sales;

	AMOUNT	HIGHEST_VALUE
1	6828	9981
2	3067	9981
3	2104	9981
4	7641	9981
5	1251	9981

-- 104. Compare each sale with lowest.

select amount,last_VALUE(amount) over(order by amount desc rows between unbounded preceding and unbounded following) as Lowest_Value from sales;

	AMOUNT	HIGHEST_VALUE
1	6828	9981
2	3067	9981
3	2104	9981
4	7641	9981
5	1251	9981

-- 105. Find highest sale per store.

select s.store_name,sa.amount,FIRST_VALUE(sa.amount) over(partition by s.store_name order by amount desc) as Highest_sale_store

from sales sa join stores s on sa.store_id=s.store_id;

	AMOUNT	HIGHEST_VALUE
1	6828	9981
2	3067	9981
3	2104	9981
4	7641	9981
5	1251	9981

-- 106. Find lowest sale per store.

select s.store_name,sa.amount,last_VALUE(sa.amount)

over(partition by s.store_name order by amount desc rows between unbounded preceding and unbounded following) as Lowest_sale_store

from sales sa join stores s on sa.store_id=s.store_id;

	STORE_NAME	AMOUNT	LOWEST_SALE_STORE
1	CMR	2432	105
2	CMR	8566	105
3	CMR	3520	105
4	CMR	5154	105
5	CMR	8522	105

-- 107. Compare category performance.

select p.category,sum(sa.amount) as Total_Revenue,

FIRST_VALUE(sum(sa.amount)) over(order by sum(sa.amount) desc) as Top_Category,

last_value(sum(sa.amount))

over(order by sum(sa.amount) desc rows between unbounded preceding and unbounded following)

as Low_Category from sales sa join products p on sa.product_id=p.product_id group by p.category;

	CATEGORY	TOTAL_REVENUE	TOP_CATEGORY	LOW_CATEGORY
1	Grocery	1270860	1381878	1220854
2	Home	1220854	1381878	1220854
3	Electronics	1242652	1381878	1220854
4	Fashion	1381878	1381878	1220854

-- 108. Find best customer per store.

select c.CUSTOMER_NAME,s.store_name,sum(sa.amount) as Total_Revenue,

first_value(c.customer_name) over(partition by s.store_name order by sum(sa.amount) desc) as Best_Customer

from sales sa join customers c on sa.customer_id=c.customer_id join stores s on sa.store_id=s.store_id

group by c.CUSTOMER_NAME,s.store_name

	CUSTOMER_NAME	STORE_NAME	TOTAL_REVENUE	BEST_CUSTOMER
1	Cust_1	CMR	36009	Cust_29
2	Cust_10	CMR	22719	Cust_29
3	Cust_11	CMR	29003	Cust_29
4	Cust_12	CMR	30465	Cust_29
5	Cust_13	CMR	26103	Cust_29

-- 109. Find worst performing category.

select p.category,sum(sa.amount) as Total_revenue,

last_value(p.category) over(order by sum(sa.amount) desc rows between unbounded preceding and unbounded following) as Worst_category

from sales sa join products p on sa.product_id=p.product_id group by p.category;

	CATEGORY	TOTAL_REVENUE	WORST_CATEGORY
1	Grocery	1270860	Home
2	Home	1220854	Home
3	Electronics	1242652	Home
4	Fashion	1381878	Home

-- 110. Analyze gap between best and worst.

select p.category,sum(sa.amount) as Total_Revenue,

first_value(sum(sa.amount)) over(order by sum(sa.amount) desc) as Best_Revenue,

last_value(sum(sa.amount)) over(order by sum(sa.amount) desc rows between unbounded preceding and unbounded following) as Worst_Revenue,

first_value(sum(sa.amount)) over(order by sum(sa.amount) desc)-

last_value(sum(sa.amount)) over(order by sum(sa.amount) desc rows between unbounded preceding and unbounded following) as gap

from sales sa join products p on sa.product_id=p.product_id group by p.category;

	CATEGORY	TOTAL_REVENUE	BEST_REVENUE	WORST_REVENUE	GAP
1	Grocery	1270860	1381878	1220854	161024
2	Home	1220854	1381878	1220854	161024
3	Electronics	1242652	1381878	1220854	161024
4	Fashion	1381878	1381878	1220854	161024

Date Analysis

-- 111. Find daily sales.

select sale_date,sum(amount) as Daily_sales from sales group by sale_date order by sale_date;

	SALE_DATE	DAILY_SALES
1	1/1/2025, 12:00:00	54543
2	1/2/2025, 12:00:00	63056
3	1/3/2025, 12:00:00	64701
4	1/4/2025, 12:00:00	71214
5	1/5/2025, 12:00:00	51742

-- 112. Find monthly sales.

select to_char(sale_date,'mm') as month,sum(amount) as Total_Revenue from sales group by to_char(sale_date,'mm') order by month;

	MONTH	TOTAL_REVENUE
1	01	1772396
2	02	1568236
3	03	1775612

-- 113. Find weekend sales.

select sale_date,sum(amount) as Weekend_sales from sales where to_char(sale_date,'dy') in('sat','sun') group by sale_date order by sale_date;

	SALE_DATE	WEEKEND_SALES
1	1/4/2025, 12:00:00	71214
2	1/5/2025, 12:00:00	51742
3	1/11/2025, 12:00:0(	47090
4	1/12/2025, 12:00:0(	50551
5	1/18/2025, 12:00:0(	59903

-- 114. Find first sale date.

select min(sale_date) as first_sale from sales;

	FIRST_SALE
1	1/1/2025, 12:00:00

-- 115. Find last sale date.

select max(sale_date) as first_sale from sales;

	FIRST_SALE
1	3/31/2025, 12:00:00 AM

-- 116. Find sales in specific month.

select sum(amount) as Monthly_sales from sales where to_char(sale_date,'mm')='03';

	MONTHLY_SALES
1	1775612

-- 117. Find sales growth month-wise.

select to_char(sale_date,'mm') as month,sum(amount) as Monthly_sales,

lag(sum(amount)) over(order by to_char(sale_date,'mm')) as prev_month,

sum(amount)-lag(sum(amount)) over(order by to_char(sale_date,'mm')) as growth from sales

group by to_char(sale_date,'mm') order by month;

	MONTH	MONTHLY_SALES	PREV_MONTH	GROWTH
1	01	1772396	*(null)*	*(null)*
2	02	1568236	1772396	-204160
3	03	1775612	1568236	207376

-- 119. Find repeat customers.

select c.customer_name,count(sale_id) as count from sales sa join customers c on sa.customer_id=c.customer_id group by c.customer_name

having count(sale_id)>1

	CUSTOMER_NAME	COUNT
1	Cust_11	20
2	Cust_12	20
3	Cust_13	20
4	Cust_14	20
5	Cust_15	20

-- 120. Find sales trend.

```
SELECT sa.sale_date,SUM(sa.amount) AS daily_sales,AVG(SUM(sa.amount)) OVER (

ORDER BY sa.sale_date ROWS BETWEEN unbounded PRECEDING AND CURRENT ROW) AS Sales_Trends FROM sales sa

GROUP BY sa.sale_date ORDER BY sa.sale_date;
```

	SALE_DATE	DAILY_SALES	SALES_TRENDS
1	1/1/2025, 12:00:00	54543	54543
2	1/2/2025, 12:00:00	63056	58799.5
3	1/3/2025, 12:00:00	64701	60766.666666666664
4	1/4/2025, 12:00:00	71214	63378.5
5	1/5/2025, 12:00:00	51742	61051.2